

MicroBlaze – Implementing the memory hierarchy

Objectives

The aim of this lab is to analyse the impact of the memory hierarchy on computation performance. The session objectives include:

1. Expanding the hardware platform by configuring Microblaze with specialised execution units and a memory hierarchy.
2. Studying the performance of various architectural features using a dense matrix multiplication example and three other kernels.

Adding DDR3 memory controller and Data Cache

The memory hierarchy to be implemented consists of a small on-chip cache and a larger off-chip DDR memory. The Zybo Z7 board features 1GB of DDR3 memory (512MB usable for this laboratory), which connects to the hardened memory controller within the processing system. The provided `add_ext_memory.tcl` script updates the ZYNQ subsystem by configuring the Zynq's memory controller to communicate with the DDR memories on the board.



Figure 1: The external memory subsystem after running the `.tcl` script. The Zynq block is encapsulated together with interconnect utilities.

Using MicroBlaze's advanced configuration tab, add a data cache with a typical configuration: 8kB capacity and a line length of eight words.

Connect the external memory to the MicroBlaze cache port and wire any open pins to complete the connections. Verify that the base and high addresses corresponds to the desirable cacheable region. Validate the design, generate the bitstream, and update the platform in Vitis with the new `.xsa` file.

Software

A naive dense matrix multiplication kernel is provided as a potential benchmarking setup. Notice that

timer.c/timer.h configures the timer in 64-bit mode for measuring execution time between two points in a C program. Use this kernel to validate your setup and run some basic tests.

Homework

The objective of the homework is to analyse the memory access patterns of kernels 1-3 and optimise them for performance (throughput). A potential strategy may be:

1. Establish a test pattern suitable for the task in terms of size, data type, and data distribution.
2. Establish a baseline with a minimal configuration, and relatively measure the impact of your optimisations on kernel performance.

Performance optimisations may be obtained by exploiting architectural features (e.g. cache configuration), manual code re-writing, compiler optimisations (automatic optimisation does not count!), or otherwise.

Kernel 1

```
void kernel1 (float *h, float *w, int *idx, int size) {  
    for (int i = 0; i < size ++i) h[idx[i]] = h[idx[i]] + w[i];  
}
```

Kernel 2

```
float kernel2 (float *A, float *B, int size) {  
    float sum = 0;  
    for (int i = 0; i < size; i++) {  
        float diff = A[i] - B[i];  
        if (diff > 0) sum = (sum + diff);  
    }  
    return sum;  
}
```

Kernel 3

```
void kernel3( float val[size], int c[size], int r[size+1], float x[size], float y[size], int size) {  
    int i, j;  
    for(i = 0; i < size; i++) {  
        for(j = r[i]; j < r[i+1]; j++) y[i] += val[j] * x[c[j]];  
    }  
}
```

Some tips that may be helpful:

- Hardware features such as the integer multiplier, floating point unit, and barrel shifter can be enabled or disabled via compiler options.

- Caches can be enabled or disabled in software.
- Before running your benchmarks consider invalidating the cache. In repeated experiments, small data arrays may already reside in the cache at the start of the experiment.

Guidelines and Deliverables:

This part of the homework contributes 15% toward the final mark and will be assessed via demonstration (June 18th, 2025) and a report to be submitted on Moodle by June 25th, 2025. A copy of the block diagram and modified source code should be included as an appendix. In addition to your report, please also submit your project (Vivado & Vitis) as a .zip file.

In the report, describe your approach to identifying the characteristics of each kernel, as well as the reasoning behind the chosen approach for improving performance. Support your reasoning quantitatively with experiments.